COMP 3981: Project Description (Part #2)

State Space Generator

General Instruction:

For this part of the project, you will work with your team members on developing the state space generator for the abalone game-playing agent. Your task is to, given any configuration of the game board, generate all the possible legal next-ply moves (in-line and side-step, for either the black or white player) as well as all the resulting game board configurations. Note that the state space generator needs to take the legality of the resulting move into consideration and that it is possible for different next-ply moves to end up with the same resulting game board configuration.

I Board Configuration

Recall that you may use any game board representation (coordinate system) within your program, however, the input and output interfaces must conform to the game board coordinate system depicted in Part #1.

II Input

For each input game board configuration, a plain text file, ***Test<#>.input***, will be provided to indicate the positions of the marbles. The file contains two (2) lines:

- 1st line:
  A single character, b or w, denoting the color of the marbles to be moved in the next ply.

- 2nd line:
  A string containing the coordinates of the marbles on the game board, as specified in (I), delimited by commas:

  $<Row_1><Col_1><color_1>, <Row_2><Col_2><color_2>, \ldots, <Row_n><Col_n><color_n>$

  where

Row_i	= {A, B, …, I}
Col_i	= {1, 2, …, 9}
$color_i$	= b for black marble or w for white marble
i	= {1, 2, …, n}, $n \leq 28$

  The marbles are ordered first by color (black before white), then by rows (alphabetically) and then by columns (numerically in an ascending order).

III Output

For each ***Test<#>.input***, two (2) plain text files, ***Test<#>.move*** and ***Test<#>.board***, are to be generated to indicate all the possible legal next-ply moves as well as all the resulting game board configurations as follows:

- ***Test<#>.move***:
  A file containing all possible legal next-ply moves, listed one move per row, for each input game board configuration. (**Note:** you can use any team-defined move notation that you submitted in Part #1 or the move notation provided in ***MoveNotation.pdf***)

- ***Test<#>.board***:
  A file containing all resulting game board configurations, listed one resulting game board configuration per row, corresponding row-by-row to the next-ply moves listed in ***Test<#>.move***. The positions of the marbles for each resulting game board configuration are to be indicated as with ***Test<#>.input***:

  $<Row_1><Col_1><color_1>, <Row_2><Col_2><color_2>, \ldots, <Row_n><Col_n><color_n>$

  where

  Row_i = {A, B, …, I}
  Col_i = {1, 2, …, 9}
  $color_i$ = b for black marble or w for white marble
  i = {1, 2, …, n}, $n \leq 28$

  The marbles are ordered first by color (black before white), then by rows (alphabetically) and then by columns (numerically in an ascending order).

IV Sample Input/Output Files

You are provided with two (2) sets of sample input/output files, ***Test1.input***, ***Test1.board***, ***Test2.input*** and ***Test2.board***, which you can download from the BCIT Learning Hub (Content | Project Material | Part 2) for debugging and verification purposes. You are not provided with ***Test<#>.move***, of which you can deduce from the provided files.

Note that your state space generator must work for any input game board configuration, as may be encountered during actual gameplay. You do not have to include self-elimination moves for the state space generator at this time (not included in the provided sample files).

You are strongly encouraged to comprehensively test your state space generator by creating additional input game board configurations and verifying the resulting game board configurations.

V Grading

Your state space generator will be tested with various input game board configurations to evaluate the correctness of your solution. For each input game board configuration, a correct match of the resulting game board configuration will earn 1 point and an incorrect (including missing or extra) resulting game board configuration will result in a 1 point deduction.

You will receive a mark of zero for any of the following:

1) your state space generator does not take the input file, ***Test<#>.input***, as specified in (II)
2) your state space generator does not output the files, ***Test<#>.move*** and ***Test<#>.board***, as specified in (III)

You are also required to indent and comment your code based on best practices - poorly indented and commented code will receive deductions, as appropriate.

Deliverables:

All work submitted is subject to the standards of conduct as specified in BCIT Policy 5104. For all deliverables, late submissions will not be accepted and you are allowed only one submission.

The deliverables, one per group, for Part #2 (BCIT Learning Hub submission) are as follows:

1) [50 marks] [Mar 21, 2024 @2359] Submit your State Space Generator report (for Part #2) which must include, at a minimum, the following:
   a. State Representation, Actions and Transition Model used
   b. Design and architecture of your State Space Generator (including how the legality of a resulting game board configuration is determined)
   c. Move Notation (team-defined) used in ***Test<#>.move***
   d. Team member contribution
   e. List of references used in the development of your State Space Generator
   f. Any additional documentation (as determined by each team) pertinent to the assessment of your deliverable

   in ***part2report_groupnumber.pdf***, to BCIT Learning Hub (Project Submission | Part 2: Report).

2) [50 marks] [Mar 21, 2024 @2359] Submit your source code and executable including detailed instructions on how to build and run the executable with additional ***Test<#>.input*** input files in ***part2sourcecode_groupnumber.zip***, to BCIT Learning Hub (Project Submission | Part 2: Source Code).
(**Note:** You will receive a mark of zero (i) if your code does not compile or execute on the (freshly-imaged) instructor terminal in the option lab (SE12-306) or (ii) you do not provide detailed instructions on how to (build and) run the executable with additional ***Test<#>.input*** input files.)

Note that there is no in-class demonstration for this part of the project.

Upcoming Project Due Dates:

Mar 28, 2024: Project Part #3 - Search Strategy and Optimization
Apr 4, 2024,
Apr 11, 2024: Project Part #4 - Tournament and Final Deliverable